

README

Setup & run instructions

Multi-Agent Incident Response System — Track 2 Capstone
github.com/bgolsen/agents-final-project

Multi-Agent Incident Response System

Capstone project, Track 2 (Multi-Agent Incident Response System). See [ARCHITECTURE.md](#) for the full design writeup, diagrams, and technical analysis, and [SCRIPT.md](#) for the demo video script. This README is setup/run instructions only. PDF versions of all three ([README.pdf](#) , [ARCHITECTURE.pdf](#) , [SCRIPT.pdf](#)) are included at the repo root for easier offline reading; regenerate them after editing the `.md` sources with `python .pdfbuild/build_pdfs.py` (requires Chrome and `npm` for mermaid-cli — see the script for details).

A simulated production incident comes in as an alert. Four **Google ADK** agents, each its own **A2A** server, collaborate under a rule-based Coordinator to triage it, diagnose root cause against a knowledge base of past incidents, plan a risk-ranked remediation, stop for **human approval**, execute the approved plan, and write a post-mortem. **n8n** fronts the system for alert intake/routing; **LangSmith** traces the full reasoning chain.

Demo video: `<add your unlisted YouTube/Loom link here before submitting>`

Prerequisites

- Python 3.11+ (developed on 3.12)
- (Optional but recommended) a [Google AI Studio API key](#) for real Gemini-driven reasoning. **Without one, the system still runs end-to-end** — every agent has a deterministic rule-based fallback (see [ARCHITECTURE.md](#) §6) that fires automatically when the LLM call fails, which is exactly how this project was developed and tested.
- (Optional) a [LangSmith API key](#) to see traces. Without one, tracing is a documented no-op (see [ARCHITECTURE.md](#) §3).
- (Optional) **n8n** if you want to run the alerting/ routing workflow rather than the CLI demo posting directly to the coordinator.

Setup

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
# macOS/Linux:
source .venv/bin/activate

pip install -r requirements.txt      # or requirements-dev.txt to also run tests
cp .env.example .env                # then fill in GOOGLE_API_KEY / LANGSMITH_API_KEY
```

Run the demo (single command)

```
python -m incident_response.run_incident
```

This starts the 4 specialist A2A servers (ports 8001-8004) and the coordinator API (port 8000) as subprocesses, lets you pick one of 3 canned incident scenarios, runs triage → diagnosis → planning, prints each agent's structured output, **stops and prompts you** to approve/reject/modify the remediation plan (the HITL checkpoint), then executes and prints the post-mortem. Processes are torn down automatically on exit. Each subprocess's own stdout/stderr (model calls, ADK's internal retry-on-validation-failure noise, etc.) is written to `logs/<agent>.log` instead of the terminal, so the demo output stays readable — check those files if something looks wrong.

Useful flags:

```
python -m incident_response.run_incident --scenario auth-cert-expiry # skip the menu
python -m incident_response.run_incident --auto-approve             # non-interactive
(scripted demo/CI)
python -m incident_response.run_incident --no-spawn                 # reuse already-
running servers
```

The three canned scenarios (`incident_response/data/scenarios.py`) each inject a different simulated failure so you can see error handling in action:

scenario_id	what's wrong	what fails once (chaos injection)
checkout-latency-spike	downstream payment-gateway timeouts cascading	the log API
auth-cert-expiry	expired TLS certificate	the knowledge-base search
payment-db-pool-exhaustion	leaked DB connections after a deploy	the metrics API, and the first remediation-execution attempt

Run the pieces individually

```
# one terminal per agent, or use scripts/start_agents.ps1 on Windows
python -m incident_response.a2a_server monitoring # :8001
python -m incident_response.a2a_server diagnostic # :8002
python -m incident_response.a2a_server remediation # :8003
python -m incident_response.a2a_server postmortem # :8004
python -m incident_response.api # :8000 (coordinator + HITL API)

# then drive it with curl, or:
python -m incident_response.run_incident --no-spawn
```

Each agent's A2A card is inspectable directly: `curl http://localhost:8001/.well-known/agent-card.json`

Coordinator API surface (also used by n8n):

Endpoint	Purpose
<code>POST /incidents {"scenario_id": " ... "}</code>	runs triage→diagnosis→planning, returns state at <code>awaiting_approval</code>
<code>GET /incidents/{id}</code>	full incident state
<code>GET /approvals/{id}</code>	lightweight poll target (used by n8n's Wait loop)
<code>POST /incidents/{id}/decision {"decision": "approve\ reject\ modify", "reviewer": " ... ", "modified_goal": " ... "}</code>	the HITL checkpoint
<code>POST /notify</code>	mock notification sink (stands in for Slack/PagerDuty)

n8n workflow (alerting & routing)

Import `n8n/incident_response_workflow.json` into a running n8n instance (Workflows → Import from File). It: receives an alert via webhook → calls the coordinator to run triage/diagnosis/planning → routes a notification by severity → acks the alert source → polls `/approvals/{id}` until a human has decided → sends a completion notification. The coordinator base URL is set once in the workflow's `Config` node (`http://localhost:8000` by default — change to `http://host.docker.internal:8000` if n8n runs in Docker). To trigger it:

```
curl -X POST http://localhost:5678/webhook/incident-alert \
-H "Content-Type: application/json" -d '{"scenario_id": "checkout-latency-spike"}
```

Then approve/reject via `POST /incidents/{id}/decision` as above (or point `run_incident.py` at `N8N_ALERT_WEBHOOK_URL` in `.env` to file the alert through n8n instead of directly).

LangSmith tracing

With `LANGSMITH_API_KEY` set in `.env`, every incident produces one nested run tree in your `LANGSMITH_PROJECT` (default `incident-response-adk`) — expand it to see the triage/diagnosis/remediation/execution/post-mortem stages in order, tagged `incident:<id>`, including which calls ran in degraded (fallback) mode.

Tests

```
pip install -r requirements-dev.txt
pytest tests/ -v
```

Covers the simulated data layer (KB ranking, chaos injection), schema validation, the deterministic fallback reasoning for all 4 agents, and the coordinator's resilience/HITL-goal-selection logic — all runnable without any API key or live agent servers.

Project layout

incident_response/	
agents/	monitoring / diagnostic / remediation / postmortem LlmAgents (each with an in-agent deterministic fallback)
a2a_server.py	launches one agent as a standalone A2A (Starlette) server
coordinator.py	orchestration, HITL gate, retry/fallback, execution
api/server.py	FastAPI surface used by n8n and the CLI demo
data/	simulated telemetry, knowledge base, remediation catalog, canned incident scenarios (with chaos injection)
schemas.py	Pydantic contracts agents hand off between each other
tracing.py	LangSmith helpers
run_incident.py	CLI demo entrypoint
n8n/incident_response_workflow.json	importable alert intake/routing workflow
tests/	pytest suite (no API key required)
ARCHITECTURE.md	full design writeup + diagrams

Known limitations

See `ARCHITECTURE.md` §9 for the full discussion (fallback ranking quality, per-process chaos budget, in-memory incident store, no auth on approvals).